

1. Add a New node at the end of the Linked List

class Node :

def __init__(self, data) :

self.data = data

self.next = None

def insert_end (head, data) :

new_node = Node (data)

if head is None :

return new_node

temp = head

while temp.next :

temp = temp.next

temp.next = new_node

return head

Example

head = Node (10)

head = insert_end (head, 20)

head = insert_end (head, 30)

Output :

10 → 20 → 30

2. Find the length of the Linked List

def length(head) :

count = 0

temp = head

```
while temp :
```

```
    count += 1
```

```
    temp = temp.next
```

```
return count
```

```
print ("length:", length(head))
```

Output:

length: 3

3. Display/print the Data

```
def display(head):
```

```
    temp = head
```

```
    while temp:
```

```
        print (temp.data, end="→")
```

```
        temp = temp.next
```

```
    print ("None")
```

```
display(head)
```

Output:-

10 → 20 → 30 → None

Example :

If the linked list contains 10, 20, 30, the display function prints :

10 → 20 → 30 → None

4. Tower of Hanoi

```
def tower (n, source, auxiliary, destination):
```

```
    if n == 1:
```

```
        print ("Move disk 1 from ", source, "to", destination)
```

```
    return
```

```
    tower (n-1, source, destination, auxiliary)
```

```
    print ("Move disk ", n, "From", source, "to", destination)
```

```
    tower (n-1, auxiliary, source, destination)
```

```
# Input
```

```
n = 3
```

```
tower (n, 'A', 'B', 'C')
```

Output

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Diagram :-



Source



Auxiliary



Destination